University of Amsterdam

Game Theory

Modeling and Simulating the Evolution of Cooperation

November 26, 2025

Students:
Yujia Liu
14806762

Wendy van Wooning
12493104

Lecturer:
Valeria Krzhizhanovskaya
Alberto Pérez de Alba Ortíz

Course:
Introduction Computational Science

1 Introduction

The Prisoner's Dilemma is a thought experiment that is used to model the dynamics between cooperation and competition in game theory. Two players, which cannot communicate and play a move at the same time, can choose to cooperate or not, and a score or 'payoff' is based on both of their moves. When opponents play each other repeatedly, called the iterated Prisoner's Dilemma, the scores over multiple rounds add up. Now, the move with the highest possible score, which would always be the best in a one-round game, might not always the most advantageous when playing multiple rounds.
Obviously, there are many different ways to approach the game, and it is easy to think of different strategies, that would result in a higher or lower payoff playing against different opponents repeatedly, where some might perform better against most other opponents than others.
As far as strategies for the Prisoner's Dilemma go, 'Tit For Tat' has shown to be the most successful strategy overall (Axelrod and Hamilton 1981). It starts out cooperative for its first move, and then simply copies whatever the opponent played in the previous round. Even though it can only tie against it's opponents at best, it wins in tournaments against multiple opponents with ultimately the highest score overall. Our goal is to find a strategy that can beat it. We will try to use a genetic algorithm to evolve a strategy that can hopefully perform better than Tit For Tat. To do that, we will also build a program that simulates games between opponents using different strategies, and come up with different strategies that players could use to try and win the game, and then simulate a whole tournament, including those strategies and Tit For Tat as our benchmark, and based on those results, try and evolve a strategy that can beat it from that. In this report, we will describe how.
Firstly, we will discuss how we set up our program and defined the model how we went about implementing it. We will also explain what strategies we developed and used in our experiments. Then, in the next two sections, we will describe how we implemented the genetic algorithm in our program, and how we used that to try to evolve a new strategy, also discussing how we tweaked the model and its parameters to try to get the best results. After that, we will present the experiments that we did, the results we got from them, and discuss those results. Finally, we will conclude our report with a short summary of what we did and what the outcomes were, and a reflection on our research.

2 Model definition and implementation

Initially, designing our model, we were on the wrong track twice. We first tried to build a simulator that would simulate games of 1 versus 1 player, which did work, but didn't allow us to implement a genetic algorithm that would help us evolve a new strategy. We still included it (file sim_1v1.py) and used it in an experiment, as it does work, but not to get to our final goal. After that, we tried to simulate multiple games at the same time somehow, between multiple neighbours, but that was also not the right track, as that was not the right way to implement evolution within the cooperation and/or competition dynamics. Then, finally, we built our current model (file gt.py) and in this section, we explain the main ideas behind it.

2.1 The Iterated Prisoner's Dilemma

The core interaction mechanism of this model is based on the Iterated Prisoner's Dilemma. We adopt the standard payoff matrix defined by Axelrod (Milgrom (1984)). To prevent alternating exploitation between strategies, this matrix satisfies both $T > R > P > S$ and $2R > T + S$.
We defined different payoff tables in our program. The main payoff values in the code are set as follows: $T = 5$, $R = 3$, $P = 1$, and $S = 0$, which are the original payoff values named in the literature. We have defined two other payoff tables as well for testing, of which we hope they will give us some interesting results when we use them in our experiments: one with a very high score for 'winning' by defecting were the other player cooperates, and one that punishes both players if they both defect.

2.2 Strategy Representation

Following the framework of Nowak and Sigmund (M. Nowak and Sigmund (1993)), we implement deterministic strategies with a memory length of 1. Each agent's strategy is encoded as a 5-bit binary gene string:
Bit 0: Initial action in round 1.
Bits 1-4: Action taken based on the outcome of the previous round.
These four bits correspond to four historical states: (C,C), (C,D), (D,C), (D,D).
This encoding constructs a search space containing 32 distinct strategies (2^5), encompassing all reactive strategies based on single-round memory, such as the classic Tit-for-Tat (00101) and Always Defect (11111).

2.3 Spatial Dynamics and Cellular Automata

To visualize the spatial evolution of strategies, we reproduced Nowak and May's spatial model (M. A. Nowak and May (1992)) by implementing a one-dimensional cellular automaton with periodic boundaries.
The system defaults to 50 agents (parameters can be adjusted). Each agent is in one color which represents one strategy. At the beginning, the system randomly assigns a strategy to each agent. In each generation, agents engage in games with their left and right neighbors. The update rule follows deterministic imitation dynamics: an agent adopts its neighbor's strategy in the next generation if that neighbor (including itself) has the highest score.
This local competition mechanism is the key reason behind the emergence of cooperative clusters and complex patterns in GUI.

2.4 Strategies

Next to the strategy we used as a benchmark, Tit For Tat, the one that cooperates initially but after the first move basically copies the opponent's moves, meaning that it retaliates in the next move for an opponent's defection, but also forgives when an opponent cooperates, we defined ten other strategies.
We describe them shortly in the table below, and they are also found in the code of the main program gt.py, where the rules for each strategy, their 5-bit binary gene string is also found, so they could be used in our model.

AllC	Always Cooperates.
AllD	Always Defects.
Vengeful	Cooperates until opponent defects once, then never forgives (always defects).
Ambitious	Repeats move if payoff was high, switches if payoff was low.
NegativeTFT	Suspicious Tit-For-Tat. Same behavior as TfT but starts by Defecting.
Opposing	Defects against cooperators, but cooperates if punished by defection.
Mean	Starts nice, but occasionally defects to test for exploitability
Forgiving	Retaliates, but attempts to restore cooperation after mutual defection.
Alternating	Follows a fixed cycle regardless of the opponent's actions.
Diversifying	Switches action when winning, keeps action when losing.

3 Genetic algorithm setup and implementation

After incorporating the previous ideas into the model, and defining our strategies, we could test with them a bit to see if our model worked well enough to start implementing the genetic algorithm and try and evolve a good strategy to possibly beat our designed ones, and of course the benchmark Tit For Tat. Below, we describe the components of our implementation.

3.1 Algorithm Design

While the CA component explores spatial dynamics, the genetic algorithm (GA) component aims to perform global optimization. Our implementation references Axelrod's experimental design (Axelrod et al. (1987)): the goal of evolution is not co-evolution within the population, but finding optimal strategies for a fixed, diverse environment.

3.2 Fitness Function

An individual's fitness is defined as its total score after a number of rounds (our standard for testing was 200 rounds) of tournaments against a representative set (`FIXED_ENV_STRATEGIES` in the code) comprising 11 fixed strategies, the 10 we designed ourselves plus the benchmark Tit For Tat.
This design ensures the evolved optimal strategy possesses high robustness: it must simultaneously exploit the weak, cooperate with cooperators, and defend against defectors.

3.3 Evolutionary Operators

The GA process runs with the main steps as follows:

3.3.1 Selection

A mechanism combining elite retention is employed. The top-performing elite individuals in each generation are automatically retained for the next generation, preventing loss of optimal solutions.

3.3.2 Crossover

For two parent genes, crossover points are randomly selected to generate offspring, allowing recombination of opening moves and response patterns.

3.3.3 Mutation

We decided to use a mutation rate that was set to 0.05. (We found this rate to be the best during the finetuning.) This rate preserves genetic diversity and prevents the algorithm from prematurely converging to local optima.

4 Tuning the model and the genetic algorithm.

To create the best model and implement the genetic algorithm in the best way, we tried a couple of different values for each of the following parameters. Some of these we derived from what seemed to make sense, and then testing if it worked as we thought, as in the first parameter below, population size, for which a too large or too small size was not a logical choice, so we used a logical starting point and tested for the optimal one. For some other parameters, we derived a starting point from the literature, and then started trying different values around those ones, seeing which one gave the best result. In the rest of this section, we describe which values we chose for each parameter.

Population Size ($N = 20$) Considering the limited strategy space ($2^5 = 32$), a massive population is unnecessary. A size of 20 individuals is sufficient to provide initial diversity without significant computational overhead.

Generations (50) Defines the duration of the evolutionary process, representing how many iterations the population evolves through the tournament. We chose 50 as our standard number of generations for our experiments.

Mutation Rate (0.05) We selected a 5% mutation rate. A rate that is too low causes premature convergence to suboptimal strategies, while a rate that is too high disrupts complex behavioral patterns, degrading the Genetic Algorithm (GA) to a random search.

Rounds (200) The game length must be sufficiently long to ensure the "shadow of the future" promotes cooperation. We selected 200 rounds to minimize the influence of initial actions and maximize the importance of reaction genes for our standard first experiments. We have also done experiments which had a considerably shorter game length, 50 rounds, and with much longer games, of 500 rounds, to see if that would make a difference in the results.

Elite Count (5) The top 5 individuals are copied unchanged to the next generation. Without an elite strategy, a perfect strategy emerging in the 10th generation could be lost in the 11th due to accidental mutation or crossover selection. This mechanism prevents the Best Fitness curve from fluctuating erratically and ensures stability.

5 Experiments and analysis

As described above, we firstly did a number of experiments on simulating a tournament between our strategies, first with the standard values we selected, the basic payoff table and 200 rounds per game, of which the results are visible in Figure 1. We also did the same tournament, with two other payoff tables and two other numbers of rounds per game in the tournament. That resulted in 9 graphs, which can be seen together in Appendix A.

In Figure 1, it shows that the winner of the tournament, the way we set it up, is the so-called 'Vengeful'-strategy, which cooperates initially but as soon as the opponent defects, will from then on always defect. It does not have the 'forgiving' aspect that Tit For Tat has, which is supposed to be one of the factors of why Tit For Tat is such a successful strategy. Tit For Tat came in fourth place, which was also very surprising. Analyzing the strategies that did best, not only the Vengeful one but also the strategy that came in second, the one that only plays 'Defect', shows that the way we set up the tournament clearly favors defecting extremely highly. Of course, in the payoff table, defecting when the other player cooperates gives the most points. However, if your opponent also defects, and a lot of strategies do retaliate, you get less points than if you decide to cooperate together. Apparently, the strategies we used or the way we set up the tournament clearly favored always defecting, and trying to cooperate with your opponent was not the best strategy here, which is usually the case in the literature, where forgiving your opponent and trying to cooperate is most effective overall. The reason Vengeful then came up on top, over the one that always defects by default, is that Vengeful cooperates initially, and keeps doing that as long as the opponent never defects. This means, that playing against opponents

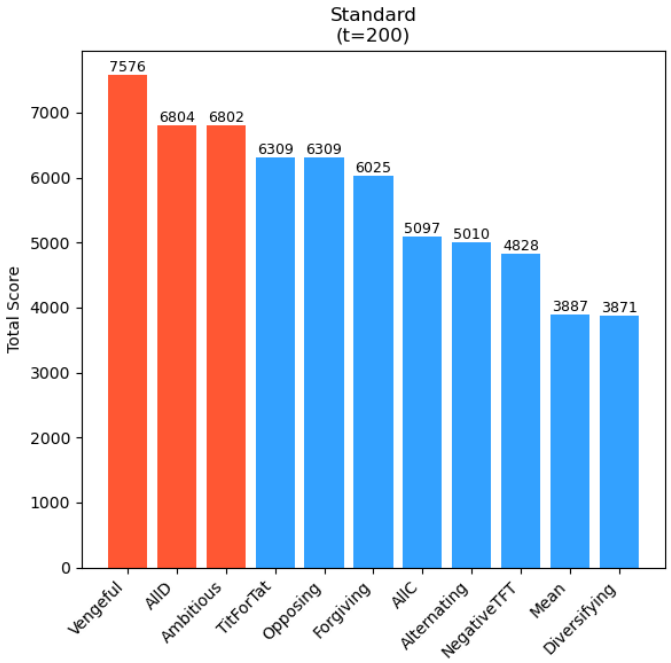

Figure 1: Results for the original payoff table for 200 rounds

that only retaliate when the player defects, both players will never defect, for example, if Vengeful plays against Tit For Tat. So, it will get more points together with Tit For Tat compared to the one that always defects, because that one does win against Tit For Tat, just because it gets more points in the first round, but will overall get a lot less points, as from then on, they will both get a lot less points than Vengeful playing Tit For Tat, who both keep cooperating. In fact, the one always defecting will win against Vengeful, as it defects in the first round where Vengeful tries cooperation, but it gets less points throughout the tournament due to this inability to cooperate, which gives Vengeful a lot of points when it meets a cooperative opponents such as Tit For Tat. That is probably why it won in this setup, with these strategies.
We tried this for other numbers of rounds, 50 and 500, and also for two different payoff tables called 'High Temptation' and 'Severe Punishment', to see if it would make a difference. All nine possibilities are shown together in Appendix A. From those nine figures, it is easy to see that not much changed. The number of rounds was no factor, but there were some differences between different payoff tables. For the payoff table that punished for both players defecting by giving them no points, Vengeful still came out on top, but the ranking behind it was a bit different. In the payoff table that highly rewarded defecting when your opponent cooperated, Vengeful only came in second, after the strategy that only defects, because Vengeful tries to cooperate initially, which just gives 'AllD' the chance to come out on top, as it never even tries and so gets a few extra points every time an opponents starts out with cooperation. But Vengeful still does very, very well, and the best strategy there is an extremely hostile one, so the main point of why we think Vengeful does so well, compared to Tit For Tat, remains standing, even though the results fluctuate slightly between different payoff tables.
Then, we did some experiments with trying to evolve a strategy using the genetic algorithm. We visualized the results of the experiment we did using the standard setup, as described in section 4, in Figure 2, which shows the fitness through the generations. The fitness gets to its peak range very quickly, and then stays around that value, with some small peaks that then make up the peak value, and some small dips, but after just about 3-5 generations, the model gets to its highest fitness range and doesn't leave it anymore.

The program returns the rule table of the best strategy it found, which, as can be seen also in the gt.ipynb file, is the rule [0, 0, 1, 1, 1], or, in words: cooperate initially, as long as the other also cooperates, but as soon as the other defects, always defect. Which is the description of the

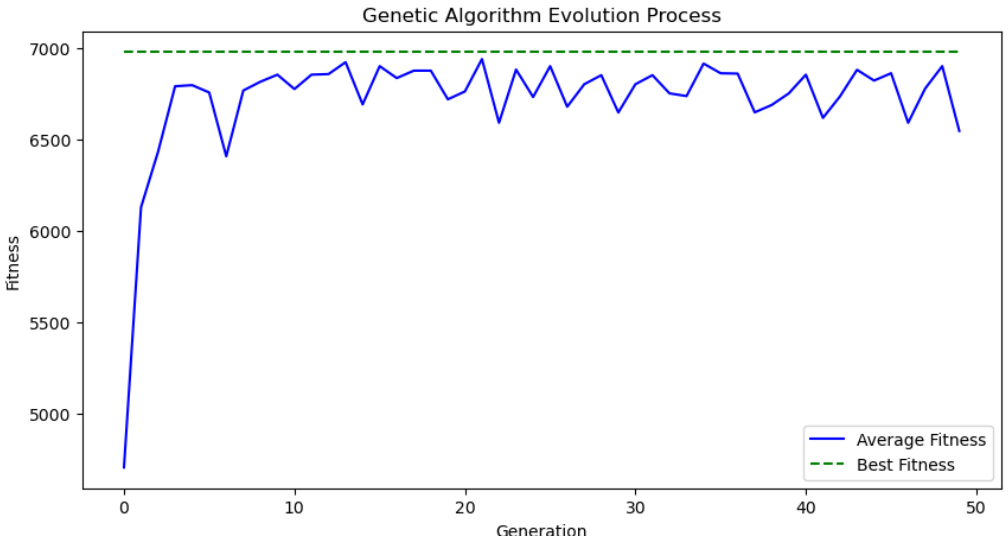

Figure 2: Graph depicting the evolution process of the genetic algorithm

Vengeful-strategy we defined earlier and was most successful in our earlier experiments as well. We tried a few different numbers for the number of generations and rounds, but that didn't make a difference. The way we set up our experiments turned out to favor the Vengeful strategy every time, and we weren't able to evolve one with our model that was more successful. Maybe our strategies that we developed are too random, meaning they try to switch moves too often regardless of the opponent's strategy, losing points when cooperating at random, which means a strategy that defects often will win from a forgiving one like Tit For Tat, that gets punished if an opponents starts defecting again. That is probably why Vengeful always turns out superior, but it is interesting to see, as we expected Tit For Tat to perform best in every situation, so also in our setup, but it clearly didn't.
As a final experiment, a sort of bonus, we decided to do some testing with the 1v1-simulator that we built at the beginning, before we realized we were on the wrong track. It simulates individual games between two strategies, and for the simulator, we first developed 12 strategies next to Tit For Tat. Some are the same as the ones we ended up using for the main program, as they were quantifiable as 5-bit strings, but some strategies were defined a bit more 'organically', like based on memory or calculations instead of rule tables.
When we found from the experiments with our main program that Vengeful was overall the most successful, as it was one of the ones defined for the 1v1-simulator, we wanted to test it out against those strategies as well. We simulated a tournament between those, for 100 rounds for each game, and the results are in the table below:

Strategy number	Description	Total Score
0	Tit For Tat	2897
1	TFT-opposed	2585
2	Always Cooperate	2232
3	Always Defect	2864
4	Alternating	2715
5	Vengeful	3525
6	Forgiving	1876
7	Diversifying	2883
8	Calculating	2863
9	Strategic	2794
10	Unpredictable	2334
11	Forgiving TFT	2859
12	Random	2483

As can be seen from the table, Tit For Tat lost to Vengeful here as well. This was more of an experiment to the side, not really relevant to the main experiments we did, but we tried it to see what the results would be and got the same result from this completely different setup as well. This was unexpected, which is why we included this as a final note in this section and why we decided to include the 1v1-simulator in the end (even though it was basically a failed first attempt at the implementation of our model).

6 Conclusion

Looking back, we got some very interesting results from our model and the experiments we did, even though it took us a while to get there. We were on the wrong track with our implementation multiple times, which set us back a lot initially and cost us a lot of time, which we would have preferred to put into further tweaking our model and doing more experiments with more different parameters, to see if that would have changed something in our results.
Our results were now very one-sided and very clearly favored the Vengeful strategy which kept coming up. From the literature, we know that a forgiving strategy usually performs best, so we would have liked to further experiment on ways to encourage cooperation and forgiveness in our model, which in our setup does not get rewarded at all, while we know it should be. In our analysis, we did give some reasons for why, in our implementation, defecting is so much more advantageous, but it would have been interesting to do experiments 'outside our model', like by partly rebuilding it, to see if we would get different results then, but we did not have quite enough time after doing a lot of work on implementations that turned out to not work for getting to our goal.
However, we are still happy with the model that we built and the experiments that we were able to do. At the start, we expected Tit For Tat to perform best and hoped to evolve a strategy that would then possibly outperform that one, but our experiments turned out very different, and that was very fascinating and allowed for an interesting analysis on why Tit For Tat just didn't seem to win from our strategies. Apparently, being forgiving is not always superior; there are some environments where playing hostile and defensive is better, like when opponents seem to be unpredictable and change their strategy a lot, the payoff is better when you stay defensive and careful. So there clearly are exceptions to the case were the aspects of Tit For Tat make it always perform the best. In different situations, different kinds of strategies can come on top, which is what our experiments and results show.
Overall, we were very satisfied with all of our results. It was our goal at the beginning to find a strategy that could beat Tit For Tat. And in a way, we did. We built our model and our strategies in such a way, that another strategy came up on top. If we wouldn't have started out wrong, we would have had more time for this implementation and results might have turned out different, but at least we got some very interesting results this way and got to learn a lot from the analysis of them. We found an environment in which Tit For Tat doesn't perform the best overall and so, we could find a strategy that beats it.
In an unconventional way, we reached our goal. And if this was a game, it didn't matter how we did it, but we would have won in the end.

References

Axelrod, Robert et al. (1987). "The evolution of strategies in the iterated prisoner's dilemma". In: *The dynamics of norms* 1.1.

Axelrod, Robert and William D. Hamilton (1981). "The Evolution of Cooperation". In: *Science* 211.4489, pp. 1390–1396.

Milgrom, Paul R (1984). *Axelrod's" The Evolution of Cooperation"*.

Nowak, Martin and Karl Sigmund (1993). "A strategy of win-stay, lose-shift that outperforms tit-for-tat in the Prisoner's Dilemma game". In: *Nature* 364.6432, pp. 56–58.

Nowak, Martin A and Robert M May (1992). "Evolutionary games and spatial chaos". In: *nature* 359.6398, pp. 826–829.

A Results tournament simulation